

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Operating Systems

COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO5: *Analyze the components and functionalities of Linux operating systems and virtualization environments to enhance their operational efficiency*

Assessment Tool Weightage: 35%

Dr VIMAL V R

QUESTIONS: 5

Total Marks:25

Annexure A

CONSTRAINT BASED PROBLEM SOLVING

Code of Conduct:

I _Narmatha J (192521341)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Narmatha J

Annexure A

CONSTRAINT BASED PROBLEM SOLVING

S.No.	Question	Mark
1	A server with a total storage capacity of 50 GB needs to effectively manage space for rapidly growing system logs, user files, and application data. Analyze the scenario and design an optimized Linux file system layout, incorporating suitable partitioning techniques and storage management tools (such as log rotation), to ensure efficient utilization of disk space and maintain system	6

	stability. The solution must ensure that log files do not exceed 10 GB and that critical services continue to operate without interruption due to disk space exhaustion.	
2	A Linux system handling high CPU workloads suffers from frequent context switching delays. Given that hardware upgrades are not possible and real-time processing must be supported, evaluate kernel-level optimization techniques such as scheduler tuning and the use of kernel modules—to improve overall system performance.	5
3	An administrator must process 1 million log entries per day using only shell scripting. Analyze the problem and design an efficient solution using optimized Linux commands that ensures the script completes within 2 minutes. Justify the choice of commands used for achieving high performance	5
4	A Linux system runs multiple background services, where a faulty process is consuming 90% of CPU resources. Without restarting the system and while ensuring critical services remain unaffected, analyze the situation and propose a process management strategy using appropriate Linux tools to identify, monitor, and control the faulty process.	5
5	A physical server must host 5 virtual machines with a total available RAM of 16 GB. Each VM requires a minimum of 2 GB, while the host operating system needs 4 GB. Analyze the scenario and propose an efficient resource allocation strategy using virtualization concepts to ensure all virtual machines operate smoothly under the given constraints.	4

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

ANSWERS:

1. Optimized Linux File System Layout – 50 GB

Given Constraints:

- Total storage = 50 GB
- Log files must not exceed 10 GB
- System logs, user files and application data must be stored.
- Critical services should continue even when storage becomes full.

Proposed Layout:

Partition	Size	Purpose
/	12 GB	Linux OS and system files
/var	10 GB	Logs, cache and variable data
/home	15 GB	User files
/opt	8 GB	Application data
Swap	4 GB	Virtual memory
Free reserve	1 GB	Emergency space

Total = 50 GB

Solution:

1. Keep /var as a separate partition so that growing log files cannot completely fill the root filesystem.
2. Set a maximum practical limit of 10 GB for logs.
3. Use logrotate to rotate old logs automatically.
4. Compress older logs using tools such as gzip.
5. Delete logs older than a specified retention period.
6. Monitor disk usage using:
 - df -h
 - du -sh

7. Configure alerts when /var reaches around 80–90% usage.
8. Keep some free space as an emergency reserve so critical services can continue operating.

Conclusion:

Separating /var, limiting logs to 10 GB, and using logrotate prevents log growth from exhausting the entire 50 GB disk and improves system stability.

2. Reducing Context Switching in a High-CPU Linux System**Constraints:**

- CPU workload is high.
- Frequent context switching causes delays.
- Hardware upgrades are not possible.
- Real-time processing must be supported.

Proposed Solution:

1. Tune the CPU scheduler to give important real-time processes higher priority.
2. Use Linux scheduling policies such as SCHED_FIFO or SCHED_RR for suitable real-time tasks.
3. Use CPU affinity with taskset to keep important processes on selected CPU cores.
4. Reduce unnecessary background processes to decrease scheduler workload.
5. Use kernel modules only when required, avoiding unnecessary modules that consume system resources.
6. Monitor context switches using tools such as vmstat, pidstat, and /proc.
7. Use appropriate kernel parameters to reduce unnecessary scheduling overhead.

Example:

```
taskset -c 2 ./realtime_program
```

This restricts the program to CPU core 2, reducing unnecessary movement between CPUs.

Conclusion:

Scheduler tuning, CPU affinity, real-time priorities, and removal of unnecessary background work can reduce context-switching overhead without requiring hardware upgrades.

3. Processing 1 Million Log Entries Using Shell Script**Constraint:**

The system must process 1 million log entries per day and the script must finish within 2 minutes.

Efficient Approach:

Avoid processing each line using a shell while read loop because starting multiple commands repeatedly can be slow.

Use optimized Linux commands such as:

- grep – filtering
- awk – processing and counting
- sort – sorting
- uniq – removing/counting duplicates
- cut – extracting fields

Example:

```
awk '{print $1}' access.log | sort | uniq -c | sort -nr > result.txt
```

This extracts the required field, counts occurrences and sorts the results.

Why these commands?

1. AWK processes large files efficiently in a single operation.
2. Grep quickly removes unwanted entries.
3. Sort and uniq efficiently handle large amounts of text.

4. Avoiding unnecessary loops reduces process creation overhead.
5. Processing the log as a stream reduces memory usage.

Optimization:

For large files, combine operations where possible instead of repeatedly reading the same file.

Conclusion: Using native Linux text-processing utilities instead of line-by-line shell loops provides much better performance and makes achieving the 2-minute constraint more realistic.

4. Managing a Faulty Process Using 90% CPU**Given Constraints:**

- One process consumes 90% CPU.
- System must not be restarted.
- Critical services must remain unaffected.

Proposed Strategy:

Step 1: Identify the process

`top`

or

`ps aux --sort=-%cpu`

These commands identify processes consuming high CPU.

Step 2: Monitor it

Use:

`top -p PID`

or:

`pidstat -p PID 5`

This helps determine whether CPU usage is continuously high.

Step 3: Reduce its priority

`renice 10 -p PID`

This gives the faulty process lower CPU scheduling priority.

Step 4: Control the process

If it is still affecting the system:

kill PID

If it does not respond:

kill -9 PID

The forceful option should be used only as a last resort.

Step 5: Protect critical services

Critical services should be monitored separately and should not be terminated. If necessary, resource-control mechanisms such as systemd resource limits/cgroups can restrict CPU usage of the faulty service.

Conclusion:

Identify → monitor → reduce priority → limit resources → terminate only if necessary. This controls the faulty process without rebooting and protects critical services.

5. Allocating 16 GB RAM Among 5 VMs

Given Constraints:

- Total physical RAM = 16 GB
- Host OS requires = 4 GB
- 5 VMs
- Each VM requires minimum = 2 GB

Calculate Available RAM:

RAM available for VMs:

$$16 \text{ GB} - 4 \text{ GB} = 12 \text{ GB}$$

Minimum required by 5 VMs:

$$5 \times 2 \text{ GB} = 10 \text{ GB}$$

Therefore:

$$\text{Remaining RAM} = 12 - 10 = 2 \text{ GB}$$

Proposed Allocation:

Component	RAM
Host OS	4 GB
VM1	2 GB
VM2	2 GB
VM3	2 GB
VM4	2 GB
VM5	2 GB
Remaining	2 GB

Efficient Strategy:

Give each VM its required 2 GB minimum, keeping 2 GB as reserve memory for temporary workload increases.

The hypervisor can use virtualization techniques such as:

- Dynamic memory allocation
- Memory ballooning
- Memory overcommitment, if supported and carefully controlled
- Monitoring VM memory usage

The host should always retain its required 4 GB so that virtualization and management services remain stable.

Conclusion:

The safest allocation is 4 GB for the host + 2 GB for each VM = 14 GB, leaving 2 GB as a reserve. This satisfies the minimum requirement of every VM while maintaining host stability.

